

Worksheet — 1.4 Data Types, Data Structures & Boolean Algebra

Name: _____ Date: ____

OCR A Level Computer Science (H446) · Component 01 · 1.4.1 Data types · 1.4.2 Data structures · 1.4.3 Boolean algebra

Time: ~40 min. SHOW ALL WORKING.

Activity 1 — Conversion drill (denary ↔ binary ↔ hex)

All binary values are **8-bit unsigned**. Fill in every blank cell. Use the place-value headers to help.

| 128 | 64 | 32 | 16 | 8 | 4 | 2 | 1 |

#	Denary	Binary (8-bit)	Hex
1	217		
2		10101101	
3			5A
4	202		
5		00101111	
6	190		
7			2D
8		10110011	

Working space:

Activity 2 — Two's complement (8-bit signed, range -128 to +127)

Reminder: to find -x, invert ALL bits then ADD 1.

(a) Represent each of these denary values in 8-bit two's complement:

Denary	8-bit two's complement
-20	
-100	
-1	

Working:

(b) Carry out these subtractions using **A + (-B)** in 8-bit two's complement. Show A, the two's complement of B, the addition, and **discard the final carry** out of the MSB. State the denary result.

(i) 35 - 18

A = _____
 -B = _____ (two's complement of 18)

 Sum = _____ (carry out: ____ → discard)

Denary result = __

(ii) 20 - 50

A = _____
 -B = _____ (two's complement of 50)

 Sum = _____

Denary result = __ (Is the result negative? How does the MSB tell you?) _____

Activity 3 — Floating-point normalisation

A floating-point number is stored as **value = mantissa × 2^{exponent}**, with the mantissa and exponent both in **two's complement**, and the binary point assumed **after the first (sign) bit of the mantissa**.

Normalisation rule: a positive number's mantissa must start **0.1...**; a negative number's must start **1.0...**.

You are given the **un-normalised** positive mantissa **0.0001101** (binary point after the first bit). Normalise it.

(a) How many places, and in which direction, must the point move so the mantissa starts **0.1...**? _____

(b) Moving the point that direction — does the exponent **increase** or **decrease**? _____

(c) Write the normalised mantissa: _____

(d) Write the exponent (in denary) needed so the value is unchanged: _____

(e) In one sentence, why do we normalise? _____

Activity 4 — Binary shifts

For each, give the **resulting 8-bit pattern** AND state the **operation/effect** (direction and $\times$ or $\div 2^n$). Bits shifted off the end are lost.

(a) 00010110 shifted **logical left by 2**: Result = __ Effect = _____

(b) 10110000 shifted **logical right by 3**: Result = __ Effect = _____

Activity 5 — Data structures

(a) **Match** each structure (1–10) to its description (A–J). Write the letter.

Structure	Letter
1. Array	
2. Record	
3. List	
4. Tuple	
5. Linked list	
6. Stack	
7. Queue	
8. Tree	
9. BST	
10. Hash table	

Descriptions: - **A.** FIFO; enqueue at the rear, dequeue at the front. - **B.** Fixed-size, same-type collection accessed by index (0-based), $O(1)$ access. - **C.** Ordered, **immutable** sequence — cannot be changed once created. - **D.** Connected, acyclic structure with a root, parents, children and leaves. - **E.** Keys mapped to indices via a hash function for $\sim O(1)$ lookup; two keys to one index = a collision. - **F.** Nodes of data + a pointer to the next; head pointer, last points to null; no random access. - **G.** Collection of named fields that may hold **different** types. - **H.** LIFO; push/pop/peek at the top via a stack pointer. - **I.** Binary tree where the left subtree $<$ node $<$ right subtree. - **J.** Ordered, **mutable**, resizable collection; append/insert/remove/index.

(b) **Trace table.** Start with an **empty stack** and an **empty queue**. Apply the operations in order to BOTH structures (push/enqueue the same value, pop/dequeue when shown). Record the contents after each step and the value removed.

Operations in order: **add 7, add 3, add 9, remove, add 5, remove.**

Step	Operation	Stack contents (top → bottom)	Stack removed	Queue contents (front → rear)	Queue removed
1	add 7		–		–
2	add 3		–		–
3	add 9		–		–
4	remove				
5	add 5		–		–
6	remove				

Activity 6 — Boolean algebra & logic

(a) Complete the truth tables.

XOR ($Q = A \oplus B$)

A	B	Q
0	0	
0	1	
1	0	
1	1	

NAND ($Q = \neg(A \cdot B)$)

A	B	Q
0	0	
0	1	
1	0	
1	1	

(b) Simplify using De Morgan's ("break the bar, change the sign — negate **each** variable and swap AND ↔ OR"). Show each step.

(i) $\neg(A \cdot B) + B$

(ii) $\neg(\neg A + \neg B)$

(c) **Half adder.** Complete the outputs for the two inputs A and B.

A	B	Sum ($A \oplus B$)	Carry ($A \cdot B$)
0	0		
0	1		
1	0		
1	1		

State the Boolean expressions: **Sum** = __ **Carry** = ____

Challenge box

A programmer claims that the 8-bit two's complement pattern `10000000` is "negative zero, so it equals 0."

- (a) What denary value does `10000000` actually represent in 8-bit two's complement? Show your reasoning using the -128 place value.
- (b) Explain why two's complement (unlike sign-and-magnitude) has **only one** representation of zero.
- (c) A colour is written `#1E90FF`. Convert the green channel `90` (hex) to denary, and the blue channel `FF` to denary. Why is hex used for colours rather than raw binary?